AI-Assisted Software Development

AI and how it helps Software Development

Mike Burton 4-6 November 2025

Introduction to "AI" and its place in Software Development

1. Generative AI has revolutionised many areas of work

- Including Software Development
- It is becoming an essential tool for developers
- We need to master it!

2. In this module, we will:

- Introduce Generative AI and Large Language Models
- Examine traditional developer workflows
- Explore AI-powered workflows and their recent progression

Overview of (Generative) "AI"

Why "Generative"?

1. Machine Learning gave us Neural Networks to do:

- Classification
- Recommendation
- Regression

2. Generative AI **generates:**

- Text
- Music
- Images
- Videos

Based on Large Language Models

1. Built with Transformer architecture

- From blocks of deep Neural Networks

2. Culmination of breakthroughs in:

- Neural Network techniques
  - Google's "Attention is all you need" paper
- Hardware
  - **Nvidia**, also:
  - Amazon
  - and even Apple
- Business
  - Amazon AWS, easy access to hardware
  - Microsoft, "buying" and popularising OpenAI & ChatGPT

What is a Large Language Model (LLM)?

1. Model

- Use past data to predict (guess!) the future
- Simple example: Journey plot

2. Language model

- Predict next word
- Then iterate

3. Large

- Several hundred billion parameters (like "factors")
- Enables a GOOD guess!

Lifecycle of building an LLM

1. Build

- Layers of Neural Networks
- (further reading!)

2. Train

- Feed it MANY examples of language(S!)
- eg entire internet as at a certain date
- Very compute-intensive
- Requires thousands of GPUs
- or other specialised hardware, eg AWS Trainium

Lifecycle of building an LLM...

3. Fine-Tune

- eg Tune for Chat-capability
- Feed it many Question/Answer pairs

4. Deploy

- Locally on BIG hardware
  - GPUs
  - NPUs (Apple A11.. M1.. AWS Inferon)
- or via API charged per token ("word") sent & received

Using an LLM

1. Query

- Send a "prompt"
- Receive a "completion"
- These accumulate into "context" (ie conversation)
- Answer / Reply / Completion is based on available data:
  - Training data
  - Fine-tuning data
  - Context

2. Add other data into context

- Up to "context window" size

3. Techniques

- Prompt engineering
- Retrieval Augmented Generation ("RAG")
  - For data bigger than context window
  - Identify a "similar" subset of available data
  - via similarity search using "Vector Database"
  - Add this into the conversation context

Traditional developer workflow

1. Familiar path:

- Easy
- In context
- Comfortable

2. Unusual path:

- For out of the ordinary tasks
- Familiarise, read the docs, read the code
- (Re-) Learning curve
- Productive after "warm-up period"
- Uncomfortable at first (especially if time pressured!)

Early AI developer workflow (c2023)

AI / LLM "Knows" all the docs and code samples

1. ChatGPT
2. Perplexity for AI-powered web-searches
3. Claude, OpenAI... chat consoles
4. Copy and Paste!

2025 AI developer workflow

Integrated IDE tools

1. Aware of context
   - Entire codebase or subset
   - Can add specific files, notes, websites...
2. Reasoning models can plan
   - Then execute steps of the plan
3. Not just coding
   - Also assist other areas of software lifecycle
4. Go beyond delivering information
   - Agents can perform actions

2025 AI software development tools

1. Early example Github Copilot

- AI powered code completion utilising vast number of Github codebases

2. Cursor

- Understands whole codebase
- Predicts:
  - code completion "next word" + next cursor & edit locations
- Dynamically chooses suitable LLMs
- Agent capability (including unchecked "YOLO mode")
- Privacy-first approach
  - special "No Train" agreement with OpenAI
- Still need to be aware of privacy re other LLMs
  - eg cursor-tools add-on uses Gemini

3. Claude Code

- Developed by Anthropic for their own use
- Initially Command-line tool (useful for Unix pipelines)
- Later added IDE integration for VSCode, IntelliJ and others

Getting Started

1. Simple tools

- ChatGPT
- Claude
- Perplexity

2. Prompt Engineering

- Use comprehensive prompts
- More detail yields better answers
- Avoid simple Google-style searches
- Prompts can be several pages long

3. Be mindful of

- Model capability (and cost trade-offs)
- Context Window size
- Citation capability
- Cut-off date (old or flawed training data)
- Privacy of data and flow (possibly also IPR issues?)